

LEARNING MANAGEMENT SYSTEM

Full Stack Web Application

FINAL PROJECT REPORT

Project Type	Individual Major Project
Domain	Full Stack Development
Primary Focus	Frontend + Backend + Database + Authentication + Dashboards
Student	Krushna Manthalkar
Program	MCA
Project Organization	Quillance Infotech Pvt. Ltd. — Major Project
Repository	KrushnaManthalkar/KrushnaManthalkar_LMS

Project objective: build a functional LMS in which students can register, enroll in courses, access structured learning modules, submit assignments and track progress, while administrators manage courses, modules, assignments, submissions and student progress.

Prepared for Quillance Infotech Pvt. Ltd. Major Project Submission

DECLARATION

I hereby declare that the project titled Learning Management System (LMS) is a major project developed as part of the Quillance Infotech Pvt. Ltd. Major Project. The implementation demonstrates full-stack web development concepts including frontend development, backend REST APIs, database integration, authentication, authorization, CRUD operations, dashboards, assignment workflows and progress tracking.

The report documents the functionality represented by the project repository and the Quillance Infotech Pvt. Ltd. project brief. Any future enhancements described in this report are identified as future scope rather than current functionality.

Student	Krushna Manthalkar
Program	MCA
Project Organization	Quillance Infotech Pvt. Ltd. — Major Project
Project	Learning Management System (LMS)

ACKNOWLEDGEMENT

I would like to express my gratitude to the faculty, mentors and training team who provided the project requirements and guidance for completing this Learning Management System. I also acknowledge the value of the development tools and open-source technologies used to design, implement, test and document the application.

This project provided practical exposure to frontend development, Node.js and Express backend development, MongoDB database management, JWT authentication, role-based authorization, REST APIs, validation, responsive UI design, Git/GitHub workflows and technical documentation.

ABSTRACT

The Learning Management System (LMS) is a full-stack web application designed to provide a structured digital learning environment. The project brief defines the goal as a functional LMS where students can enroll in courses, access modules, submit assignments and track progress, together with an administrator/instructor dashboard for course, assignment, submission and student-progress management.

The implemented application uses a browser-based frontend built with HTML, CSS and JavaScript, a Node.js/Express REST API backend, MongoDB with Mongoose for persistent storage, JWT for authentication and bcryptjs for password hashing. The system separates Student and Admin workflows and applies validation and authorization checks to protected operations.

The completed workflow covers registration/login, course discovery and enrollment, learning modules, module completion, progress calculation, assignment submission, deadline and duplicate-submission checks, Admin review, marks/feedback and student progress monitoring. The project also includes responsive UI work, testing and GitHub-based documentation.

Keywords: LMS, Full Stack Development, Node.js, Express.js, MongoDB, Mongoose, JWT, REST API, Role-Based Authorization, Course Management, Assignment Submission, Progress Tracking.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Requirements Analysis
6. Feasibility and Development Method
7. Technology Stack
8. System Architecture
9. User Roles and Use Cases
10. Functional Modules
11. Database Design
12. API Design
13. Security and Validation
14. Major System Workflows
15. User Interface and Responsive Design
16. Testing and Verification
17. Project Development Phases
18. Results and Expected Outcome
19. Limitations
20. Future Enhancements
21. Deployment and Maintenance
22. Project Deliverables
23. Conclusion
24. References
Appendix A — Screenshot Index
Appendix B — Viva / Demonstration Flow

1. INTRODUCTION

Learning has increasingly moved toward digital platforms that allow learners to access structured course material, complete activities and monitor progress from a single application. The project brief for this major project specifies a Learning Management System as a full-stack application focused on frontend, backend, database, authentication and dashboards.

The LMS developed in this project provides two primary workflows: a Student workflow for learning and assessment, and an Admin workflow for managing learning content and monitoring student activity. The application is designed as an academic/training project, with emphasis on demonstrating practical full-stack development rather than implementing every feature of a commercial learning platform.

The implementation follows a REST-based client/server structure. The frontend communicates with a Node.js and Express.js backend, while MongoDB provides persistent storage through Mongoose models. Authentication tokens and role middleware protect restricted operations.

The project brief also emphasizes database integration, authentication/authorization, CRUD operations, responsive pages, testing and Git/GitHub documentation. These areas are therefore treated as core parts of the final system.

1.1 Project Goal

The goal is to build a functional LMS in which students can register, enroll in courses, access learning modules, submit assignments and track progress, while administrators can manage courses, modules, assignments, submissions and student progress.

1.2 Development Context

The project was planned around the 10-day development structure supplied in the project brief: planning and requirements, UI design, backend/database setup, authentication, course management, enrollment/modules, assignment system, progress tracking, admin dashboard/testing, and final deployment/documentation.

2. PROBLEM STATEMENT

In a basic learning environment, course information, learning resources, assignments and student progress can become fragmented across separate tools. Students need a clear path from course discovery to enrollment, learning, assessment and progress tracking. Administrators need centralized control over courses and assignments and need visibility into student enrollment, submissions and progress.

The project therefore addresses the need for a centralized web application that connects these learning activities through one database-backed workflow. The system is not intended to replace a large commercial LMS; instead, it demonstrates the essential workflows defined in the academic project brief.

2.1 Existing Challenges

- Separate handling of course information and learning resources.
- Difficulty maintaining a consistent student learning workflow.
- Manual or disconnected assignment submission and review processes.
- Limited visibility into module completion and course progress.
- Need for role-specific access to administrative operations.
- Need for persistent database-backed records rather than browser-only state.

2.2 Proposed Solution

The proposed LMS provides a single web interface backed by REST APIs and MongoDB. Student actions are authenticated and associated with the logged-in user, while Admin operations are protected using role checks. Enrollment, module completion, assignment submission and review records are persisted so that the workflow remains available after refresh or application restart.

3. OBJECTIVES

The project objectives are derived from the project brief and the implemented repository.

- Provide student registration and secure login.
- Allow students to browse available courses and view course details.
- Allow students to enroll in courses without creating duplicate enrollment records.
- Provide ordered learning modules and resource links.
- Track module completion and calculate course progress.
- Provide assignment creation and management for Admin users.
- Allow students to submit assignment work through a submission link.
- Validate assignment deadlines and prevent duplicate submissions.
- Allow Admin users to review submissions, assign marks and provide feedback.
- Provide Admin visibility into students, enrollments and progress.
- Protect APIs and pages using authentication and role-based authorization.
- Validate IDs, required fields and business rules.
- Provide a responsive frontend suitable for desktop and mobile widths.
- Document the complete project through GitHub README, screenshots and final report.

4. SCOPE OF THE PROJECT

The scope is intentionally aligned with the project brief's required LMS workflow. The core scope includes authentication, course management, enrollment, modules, assignments, submissions, progress tracking, Admin dashboards and database integration. The brief identifies these as the central features of the major project.

4.1 Student Scope

- Registration and login.
- Course browsing and course details.
- Course enrollment.
- My Courses and dashboard navigation.
- Module/resource access.
- Module completion and progress tracking.
- Assignment viewing and submission.
- Marks, feedback and submission status viewing.

4.2 Admin Scope

- Admin authentication and protected dashboard.
- Course CRUD.
- Module CRUD and ordering.
- Assignment CRUD.
- Submission review.
- Student listing and progress visibility.

4.3 Out-of-Scope / Future Scope

- Advanced quizzes and automated assessments.
- Certificates and advanced analytics.
- Native file storage for learning materials and submissions.
- Advanced notifications and communication features.
- Large-scale production infrastructure and monitoring.

5. REQUIREMENTS ANALYSIS

5.1 Functional Requirements

ID	Requirement	System Response
FR-01	User registration	Create a student account with validated data.
FR-02	Authentication	Login returns an authenticated session/token.
FR-03	Course discovery	Students can list and view courses.
FR-04	Enrollment	Student can enroll in a course once.
FR-05	Modules	Admin manages modules; students access course modules.
FR-06	Progress	Completed modules update course progress.
FR-07	Assignments	Admin creates/updates/deletes assignments.
FR-08	Submission	Student submits assignment work before deadline.
FR-09	Review	Admin reviews submission with marks/feedback/status.
FR-10	Monitoring	Admin views student enrollment/progress information.

5.2 Non-Functional Requirements

Area	Requirement
Security	Passwords must not be stored as plain text; protected APIs require authentication and appropriate roles.
Validation	Invalid IDs, missing fields, duplicate operations, deadlines and marks must be handled.
Persistence	Core application data must remain available after refresh/restart.
Usability	Pages should provide clear navigation and understandable forms.
Responsiveness	Application should remain usable across desktop and mobile widths.
Maintainability	Frontend, backend routes, models and middleware should remain separated.

The project brief's testing checklist specifically requires valid registration/login, duplicate registration handling, role restrictions, CRUD, enrollment uniqueness, submission storage, progress updates, validation, responsive usability and persistence after restart.

6. FEASIBILITY AND DEVELOPMENT METHOD

6.1 Technical Feasibility

The selected stack is suitable for an academic full-stack application because HTML/CSS/JavaScript can provide the client interface, Express.js can expose REST APIs, Mongoose can simplify MongoDB access, and JWT/bcryptjs can implement authentication and password protection. The project can run locally with Node.js and MongoDB or MongoDB Atlas.

6.2 Operational Feasibility

The application is designed around familiar web workflows: login, dashboard, course cards, forms, tables and progress indicators. Student and Admin responsibilities are separated so that each role sees the operations relevant to it.

6.3 Development Method

The project follows an incremental phase-based development method. Work was completed in feature groups so that authentication, course management, module management, assignment management, submission review, student progress, security and testing could be verified separately.

Incremental Development Flow



Figure 1. Incremental development flow used for the project.

7. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	HTML5	Page structure and forms
Frontend	CSS3	Responsive and visual styling
Frontend	JavaScript	Client-side workflow and API communication
Backend	Node.js	Server-side JavaScript runtime
Backend	Express.js	REST API and routing
Database	MongoDB	Persistent document database
ODM	Mongoose	Schemas, models and database operations
Authentication	JWT	Authentication token handling
Security	bcryptjs	Password hashing
Development	VS Code / Nodemon	Development and local server workflow
Version Control	Git / GitHub	Source control and project documentation

7.1 Why This Stack

The stack keeps the application within a single JavaScript-oriented development environment while still demonstrating the separation between frontend, backend API, authentication middleware, data models and database. This makes the architecture appropriate for a major-project demonstration and for explaining the complete request-to-database lifecycle.

8. SYSTEM ARCHITECTURE

The system follows a client/server architecture. The browser renders the frontend pages and JavaScript communicates with the Express backend using HTTP/REST requests. The backend applies authentication, role checks, validation and business rules before accessing MongoDB through Mongoose.

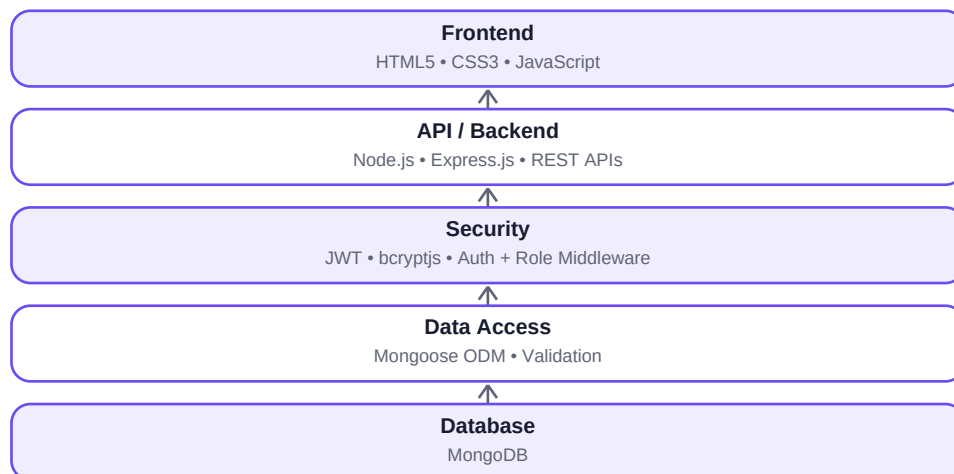


Figure 2. LMS layered system architecture.

8.1 Request Lifecycle

Typical Protected Request

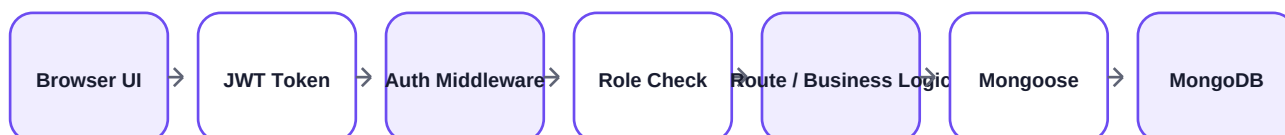


Figure 3. Typical authenticated request lifecycle.

8.2 Deployment View

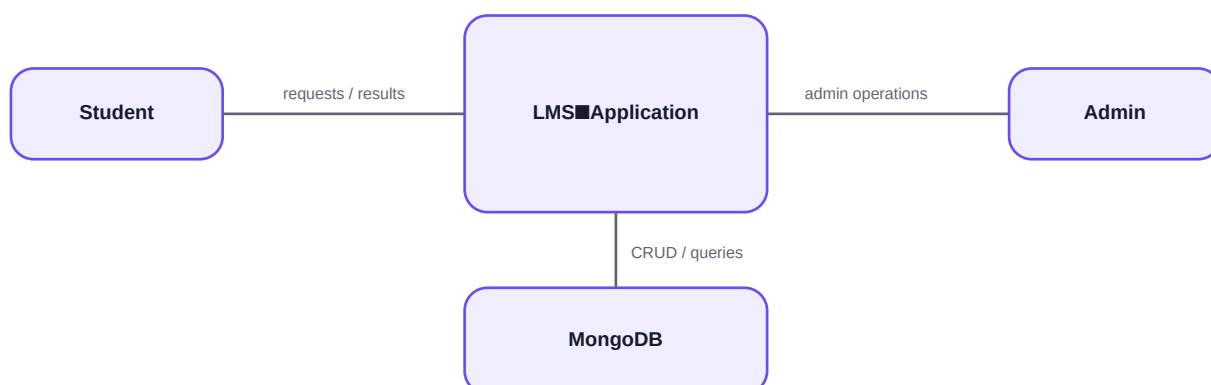


Figure 4. High-level data-flow / deployment view.

9. USER ROLES AND USE CASES

The implemented application has two primary roles: Student and Admin. Registration creates student accounts, while administrative operations are protected through role-based authorization.

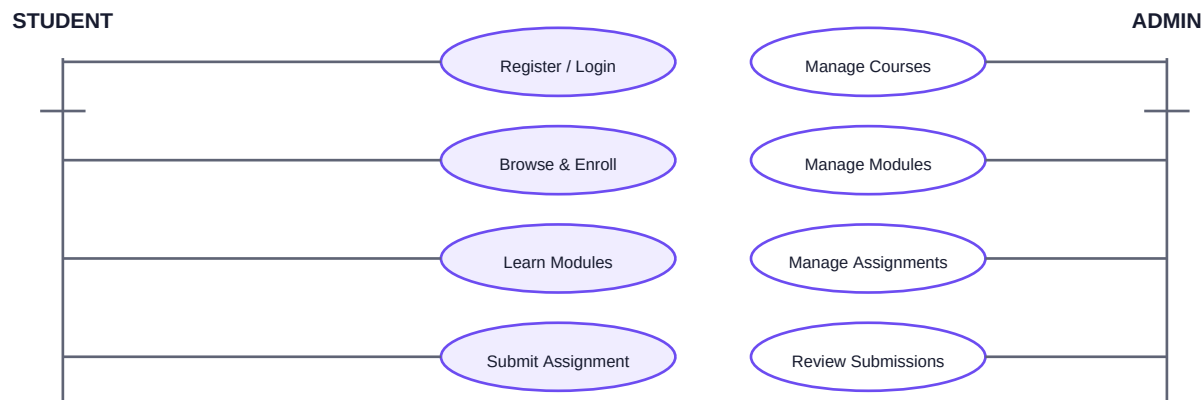


Figure 5. Major Student and Admin use cases.

9.1 Student Use Cases

- Register and log in.
- Browse courses and view course details.
- Enroll in a course.
- Open My Courses and continue learning.
- Access modules and resource links.
- Complete modules and monitor progress.
- View assignments and submit work.
- View submission history, marks, status and feedback.

9.2 Admin Use Cases

- Log in to protected Admin dashboard.
- Create, update and delete courses.
- Create, update, delete and order modules.
- Create, update and delete assignments.
- View submissions and open submitted work.
- Assign marks, feedback and review status.
- View registered students and progress information.

10. FUNCTIONAL MODULES

10.1 Authentication Module

The authentication module supports student registration, login and retrieval of the authenticated user. Passwords are hashed using bcryptjs and JWT tokens are used to identify authenticated requests. Frontend route guards and backend middleware prevent unauthorized use of restricted functionality.

10.2 Course Management

Course management is primarily an Admin responsibility. A course contains information such as title, description, category, instructor, duration, difficulty and optional image. Students can access public course listing/details and then enroll.

10.3 Enrollment and My Courses

Enrollment creates a student-course relationship. The backend prevents duplicate enrollment records. The Student dashboard and My Courses area provide navigation into enrolled learning content.

10.4 Module Management

Modules belong to a course and contain title, description, resource link and module order. Admin users can create, edit, delete and order modules. The system re-indexes ordering after deletion.

10.5 Assignment Management

Assignments belong to a course and include title, description, deadline and maximum marks. Admin users manage assignment CRUD operations, while students view assignments associated with enrolled course learning.

10.6 Submission and Review

Students submit work through a submission link. The backend verifies enrollment, checks the assignment deadline and prevents duplicate submissions. Admin users can view submissions and save marks, feedback and review status.

10.7 Progress Tracking

Module completion is persisted in the enrollment record. Progress is calculated from completed modules and total course modules. The Student interface displays the resulting progress and completion state.

10.8 Admin Students and Progress

The Admin area provides student listing and student-progress visibility, allowing administrative monitoring of enrollment and learning progress.

11. DATABASE DESIGN

MongoDB is used as the persistent database and Mongoose provides schemas and model-based access.

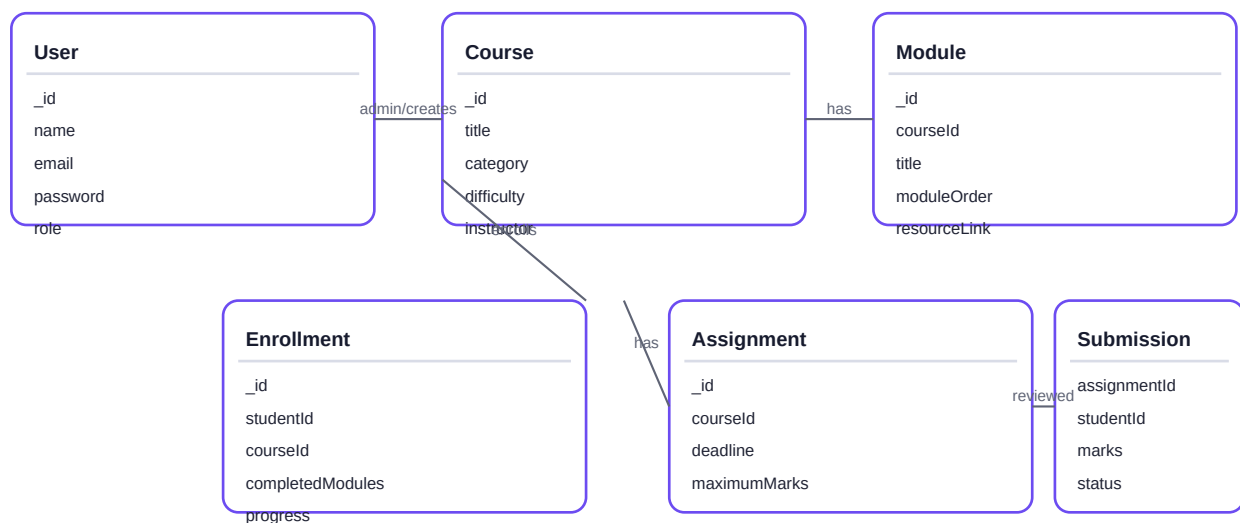


Figure 6. Simplified LMS entity/data relationship diagram.

Collection / Model	Important Fields	Purpose
User	name, email, password, role	Identity and role information
Course	title, description, category, instructor, duration, difficulty, image	Course catalogue data
Module	courseId, title, description, resourceLink, moduleOrder	Ordered learning content
Assignment	courseId, title, description, deadline, maximumMarks	Course assessment data
Enrollment	studentId, courseId, enrollmentDate, completedModules, progress, status	Student-course relationship and progress
Submission	assignmentId, studentId, submissionLink, submissionDate, marks, feedback, status	Student work and Admin review

11.1 Data Integrity Rules

- A student can enroll in a course only once.
- A student can submit an assignment only once in the implemented workflow.
- Module records are associated with a specific course.
- Assignments are associated with a specific course.
- Submission review marks are validated against maximum marks.
- Progress operations verify enrollment and course/module relationships.

12. API DESIGN

The backend exposes REST endpoints under the /api prefix. Protected routes require the appropriate authentication token and role.

Area	Representative Endpoints	Access
Authentication	POST /api/auth/register POST /api/auth/login GET /api/auth/me	Public / Auth
Courses	GET /api/courses GET /api/courses/:id POST/PUT/DELETE /api/courses/:id	Public / Admin
Enrollments	POST /api/enrollments GET /api/enrollments/my GET /api/enrollments/:courseId	Student
Modules	GET /api/modules/course/:courseId POST/PUT/DELETE /api/modules/:id	Auth / Admin
Assignments	GET /api/assignments/course/:courseId POST/PUT/DELETE /api/assignments/:id	Auth / Admin
Submissions	POST /api/submissions GET /api/submissions/my GET /api/submissions/assignment/:assignmentId PUT /api/submissions/:id	Student / Admin
Progress	GET /api/progress/:courseId POST /api/progress/:courseId/module/:moduleId/complete	Student
Admin	GET /api/admin/dashboard GET /api/admin/students GET /api/admin/student-progress	Admin

12.1 API Design Principles

- HTTP methods are used according to the operation: GET, POST, PUT and DELETE.
- Protected operations use authentication middleware.
- Role middleware separates Student and Admin functionality.
- Request IDs are validated before database operations.
- Business rules such as enrollment ownership, deadline and marks are enforced on the server.
- Errors are returned using appropriate HTTP status categories such as 400, 401, 403, 404 and 500.

13. SECURITY AND VALIDATION

Security is implemented at both the authentication and authorization layers. The backend should be treated as the authority for access control; frontend redirects improve user experience but do not replace server-side protection.

13.1 Authentication

- Registration creates student accounts.
- Passwords are hashed using bcryptjs.
- Login issues a JWT used for authenticated API requests.
- Protected endpoints require a valid authenticated identity.

13.2 Authorization

- Student-only endpoints prevent Admin-only operations from being called by students.
- Admin-only endpoints protect course/module/assignment management and review operations.
- Enrollment ownership is verified before student learning/progress operations.
- Module/course relationships are verified before progress completion.

13.3 Validation

- Required fields are validated.
- MongoDB ObjectId values are validated before use.
- Duplicate enrollment is prevented.
- Duplicate assignment submission is prevented.
- Assignment deadline is checked before submission.
- Marks are checked against maximum marks.
- Invalid or unauthorized requests return appropriate error responses.

14. MAJOR SYSTEM WORKFLOWS

14.1 Student Learning Workflow

Student Learning Journey

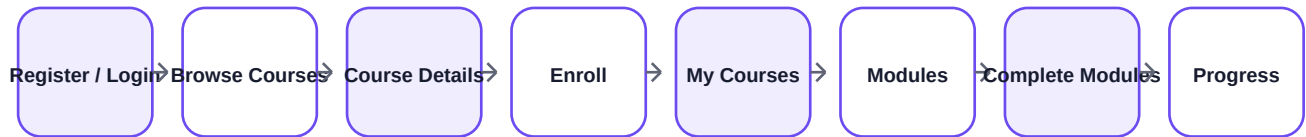


Figure 7. Student learning flow.

14.2 Assignment Workflow

Assignment Lifecycle

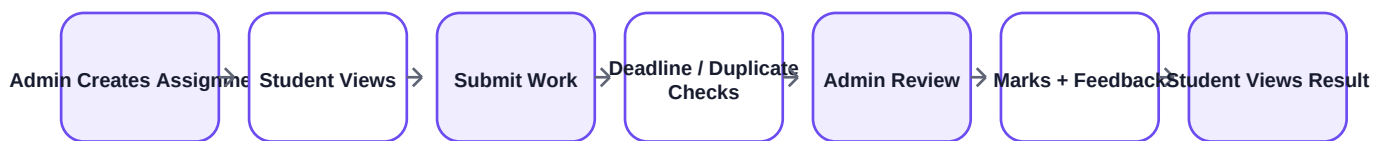


Figure 8. Assignment submission and review flow.

14.3 Progress Workflow

Progress Calculation

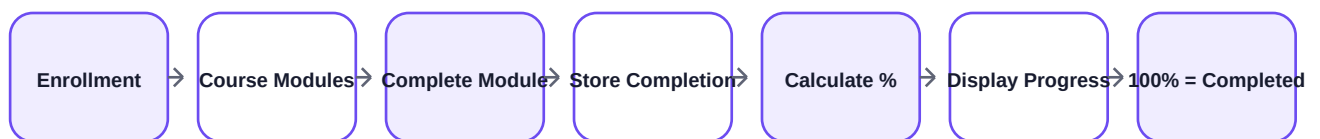


Figure 9. Progress tracking flow.

14.4 Admin Management Workflow

Admin Content Management



Figure 10. Admin management flow.

15. USER INTERFACE AND RESPONSIVE DESIGN

The frontend uses HTML5, CSS3 and JavaScript with a consistent navigation and card/form-based interface. The project includes separate pages for Student and Admin workflows.

15.1 Major Student Pages

- Login and registration
- Student dashboard
- Course catalog
- Course details
- My Courses
- Learning modules
- Assignments
- Assignment submission
- Progress
- Profile/authenticated navigation

15.2 Major Admin Pages

- Admin dashboard
- Course management
- Module management
- Assignment management
- Submission review
- Students and progress

15.3 Responsive Design

Responsive behavior was tested at desktop and mobile widths. The screenshot set includes a dedicated responsive/mobile view to demonstrate that the interface adapts to smaller screens.

15.4 Screenshot Evidence

The repository README contains the final screenshot gallery. The gallery is organized into 17 evidence images covering authentication, Student workflow, learning, assignments, progress, Admin management, submission review, student monitoring and responsive UI.

#	Screenshot Evidence
01–02	Login and Registration
03–06	Student dashboard, course catalogue, course details and My Courses
07–10	Modules, assignments, submission and progress
11–16	Admin dashboard, course/module/assignment management, review and student progress
17	Responsive mobile layout

16. TESTING AND VERIFICATION

Testing was performed against the project's major workflows and the checklist defined in the project brief. The brief requires validation of authentication, duplicate registration, role restrictions, CRUD, enrollment uniqueness, assignment submission, progress, form validation, responsive usability and database persistence.

Test Area	Verification	Expected Result
Registration	Valid student registration	Student account created
Login	Valid and invalid credentials	Valid login succeeds; invalid access rejected
Authorization	Student attempts Admin operation	Operation blocked
Course CRUD	Admin creates/edits/deletes course	Database reflects changes
Enrollment	Enroll twice	Duplicate enrollment prevented
Modules	Create/order/delete modules	Module structure and order remain consistent
Progress	Complete modules	Progress updates and persists
Assignment	Submit before deadline	Submission stored
Deadline	Submit after deadline	Submission rejected
Duplicate submission	Submit same assignment twice	Duplicate blocked
Review	Admin assigns valid marks	Marks/feedback/status saved
Invalid marks	Marks exceed maximum	Validation rejects input
Persistence	Restart backend and reload	Stored records remain
Responsive UI	Desktop/mobile widths	Pages remain usable

16.1 Error Handling

The application includes handling for invalid requests, invalid IDs, authentication failures, authorization failures, missing resources and server-side errors. This provides predictable behavior for both normal and invalid workflows.

17. PROJECT DEVELOPMENT PHASES

Phase	Focus	Outcome
1	Planning and requirements	Roles, workflow, stack and database direction
2	UI design	Home, login, registration, courses and dashboard layouts
3	Backend/database	Server, database connection, models and API routes
4	Authentication	Registration, login, password hashing, roles and logout
5	Course management	Course CRUD and course details
6	Enrollment/modules	Enrollment, My Courses, modules and resources
7	Assignment system	Assignment management and student submissions
8	Progress tracking	Module completion and progress percentage
9	Admin/testing	Statistics, role checks, validation, API testing and fixes
10	Documentation	GitHub, README, screenshots, report and presentation

The phase sequence follows the 10-day development plan in the supplied project brief.

17.1 Implemented Completion State

- Core Student and Admin workflows completed.
- Backend and database integration completed.
- Authentication and authorization completed.
- Course/module/assignment CRUD completed.
- Submission review completed.
- Progress tracking completed.
- Security and validation audit completed.
- End-to-end workflow and responsive UI testing completed.
- README and screenshot documentation prepared.

18. RESULTS AND EXPECTED OUTCOME

The completed application provides an end-to-end learning workflow that connects authentication, course enrollment, module learning, progress tracking, assignment submission and Admin review. Data is persisted in MongoDB and protected operations are handled through authentication and role-based middleware.

The result satisfies the central project goal described in the brief: students can enroll in courses, access modules, submit assignments and track progress, while Admin users can manage learning content, submissions and student progress.

18.1 Skills Demonstrated

Technical Area	Evidence in Project
Frontend Development	Responsive pages, forms, dashboards and course interfaces
Backend Development	Express API endpoints, business logic and protected routes
Database Management	Users, courses, modules, enrollments, assignments and submissions
Authentication / Authorization	JWT authentication, password hashing and role-based access
CRUD Operations	Create, read, update and delete workflows
Testing	Functional workflow, validation, persistence and responsive testing
Version Control	Git/GitHub repository and structured documentation

19. LIMITATIONS

The current system is an academic/training LMS and therefore does not attempt to implement the full feature set of a commercial learning platform.

- Learning resources and submissions are handled through links rather than a full file-storage platform.
- Advanced quizzes and automated grading are not included.
- Certificates and advanced learning analytics are not included.
- Email/SMS notification infrastructure is not included.
- Production-scale monitoring, CI/CD and enterprise deployment infrastructure are outside the current scope.
- The current Admin workflow is focused on core course, module, assignment, submission and student-progress management.

20. FUTURE ENHANCEMENTS

- Dedicated file uploads and cloud storage for learning materials and assignment submissions.
- Multiple resource types such as PDF, video, source code and exercises.
- Quizzes and automated assessments.
- Certificates after course completion.
- Course announcements and notifications.
- Email notifications and password reset.
- Search, filtering and pagination for larger datasets.
- Rich-text course/module content.
- Advanced learning analytics.
- Automated unit, integration and end-to-end testing with CI.
- Production cloud deployment with monitoring and logging.

These items are proposed future improvements and are not represented as current implemented features.

21. DEPLOYMENT AND MAINTENANCE

21.1 Local Deployment

- Install Node.js and npm.
- Install MongoDB locally or configure MongoDB Atlas.
- Clone the GitHub repository.
- Install backend dependencies using `npm install`.
- Create backend `.env` with `MONGO_URI`, `JWT_SECRET` and optional `PORT`.
- Run the backend using `npm run dev` or `npm start`.
- Serve the frontend through a local web server such as VS Code Live Server.

21.2 Environment Configuration

Variable	Purpose
MONGO_URI	MongoDB connection string
JWT_SECRET	JWT signing and verification secret
PORT	Backend port; defaults to 5000

21.3 Deployment Considerations

For public deployment, the backend must use secure environment variables, the frontend must point to the deployed API, database access must be restricted appropriately, and secrets must never be committed to Git. The project brief lists deployment as an optional live link deliverable, while GitHub documentation remains a required project artifact.

22. PROJECT DELIVERABLES

Deliverable	Project Status
Complete source code	Completed in GitHub repository
GitHub repository link	Available
Live deployment link	Optional; local execution documented
Project report	This final report
Database structure/schema	Documented in README and backend models
Screenshots of major pages	Captured and organized in README screenshot gallery
README documentation	Completed
Project presentation	Can be prepared from this report
Short project demonstration video	Can be recorded from the documented demo flow

The project brief explicitly lists these items as expected project deliverables and separately recommends README sections including overview, problem statement/objectives, key features, technology stack, architecture, installation, environment variables, database setup, screenshots, APIs, future improvements and author details.

23. CONCLUSION

The Learning Management System successfully demonstrates the core concepts expected from a full-stack major project. The application connects a responsive frontend with a Node.js/Express REST backend and MongoDB persistence, while authentication and role-based authorization protect Student and Admin workflows.

The implemented system covers the complete learning cycle from registration and course discovery to enrollment, module completion, progress tracking, assignment submission and Admin review. It also demonstrates CRUD operations, validation, duplicate protection, deadline enforcement, marks validation and database persistence.

From an academic perspective, the project provides practical evidence of frontend development, backend development, database management, authentication/authorization, REST APIs, CRUD operations, testing, responsive UI design and Git/GitHub documentation. The project therefore provides a coherent foundation for final demonstration, presentation and viva.

The system can be extended in future with richer learning resources, quizzes, certificates, notifications, analytics, cloud storage and production deployment, while preserving the current core architecture.

24. REFERENCES

- Quillance Infotech Pvt. Ltd. — Learning Management System (LMS) Full Stack Development Major Project Brief, supplied project specification.
- Project GitHub repository — [KrushnaManthalkar/KrushnaManthalkar_LMS](#).
- Node.js and Express.js project documentation and package configuration used by the implementation.
- MongoDB and Mongoose concepts used by the database layer.
- JSON Web Token (JWT) and bcryptjs concepts used for authentication and password protection.

Note: This report is grounded primarily in the supplied LMS project brief and the documented implementation represented by the LMS repository/README. Where the brief identifies optional or future features, they are not presented as implemented functionality.

APPENDIX A — SCREENSHOT INDEX

The final README screenshot gallery is organized around the major pages and workflows of the completed LMS. The report uses the screenshot index below so the final report and GitHub documentation remain consistent.

No.	Screenshot	Evidence Area
01	01-login.png	Authentication — Login
02	02-registration.png	Authentication — Registration
03	03-student-dashboard.png	Student Dashboard
04	04-course-catalog.png	Course Catalog
05	05-course-details-enrollment.png	Course Details / Enrollment
06	06-my-courses.png	My Courses
07	07-learning-modules.png	Learning Modules
08	08-assignments.png	Assignments
09	09-assignment-submission.png	Assignment Submission
10	10-student-progress.png	Progress Tracking
11	11-admin-dashboard.png	Admin Dashboard
12	12-admin-course-management.png	Admin Course Management
13	13-admin-module-management.png	Admin Module Management
14	14-admin-assignment-management.png	Admin Assignment Management
15	15-admin-submission-review.png	Admin Submission Review
16	16-admin-students-progress.png	Admin Students & Progress
17	17-responsive-mobile.png	Responsive Mobile Layout

The images are maintained separately in the repository screenshot directory and referenced by the README. This keeps the report lightweight while preserving the complete visual evidence set in GitHub.

APPENDIX B — VIVA / DEMONSTRATION FLOW

A short demonstration can be performed in the following order so that the complete system workflow is visible within a single session.

Step	Demonstration
1	Start MongoDB and backend server.
2	Open the frontend and show Login.
3	Register or log in as a Student.
4	Browse Courses and open Course Details.
5	Enroll in a course and open My Courses.
6	Open Modules and complete one module.
7	Show Progress updating.
8	Open Assignments and submit an assignment link.
9	Log out and log in as Admin.
10	Show Admin Dashboard statistics.
11	Show Course / Module / Assignment management.
12	Open the student's submission, assign marks and feedback.
13	Return to Student and show the reviewed submission.
14	Demonstrate responsive/mobile layout if requested.

This flow directly demonstrates the core project goal and the major testing areas defined in the project brief.

END OF REPORT